Uma **interface funcional** em Java é uma interface que possui **exatamente um método abstrato** (um método sem implementação própria). [1, 2]

Introduzidas no Java 8, elas são a base para o uso de **expressões lambda** e referências de métodos, permitindo escrever códigos mais limpos e com estilo de programação funcional. [1, 2, 3]

Principais Características

- **Regra do método único:** Podem ter vários métodos `default` ou `static` (que já possuem implementação), mas só **um** método abstrato. [1, 2]
- **Anotação `@FunctionalInterface`:** Opcional, mas recomendada. Ela avisa o compilador para gerar um erro caso alguém adicione um segundo método abstrato por engano. [1, 2]
- **Compatibilidade com Lambdas:** Como a interface tem só um método abstrato, o Java sabe exatamente qual método a expressão lambda está implementando. [1, 2]

Exemplos da Biblioteca Padrão (`java.util.function`)

O Java já vem com várias interfaces funcionais prontas no pacote `java.util.function`, como: [1]

- **`Predicate<T>`:** Recebe um argumento e retorna um booleano (`true` ou `false`).
- **`Consumer<T>`:** Recebe um argumento e não retorna nada (apenas consome a informação, como em um `System.out.println`).
- **`Function<T, R>`:** Recebe um argumento do tipo `T` e retorna um resultado do tipo `R` .
- **`Supplier<T>`:** Não recebe argumentos e fornece/retorna um objeto do tipo `T` .

Exemplo clássico já existente antes do Java 8 que virou interface funcional:

- **`Runnable`:** Possui apenas o método `run()` .

Exemplo Prático com Lambda

java

```java
@FunctionalInterface
public interface Calculadora {
    int calcular(int a, int b);
}

// Usando expressão lambda para implementar a interface
```

```
Calculadora somar = (a, b) -> a + b;
int resultado = somar.calcular(5, 3); // Retorna 8
```